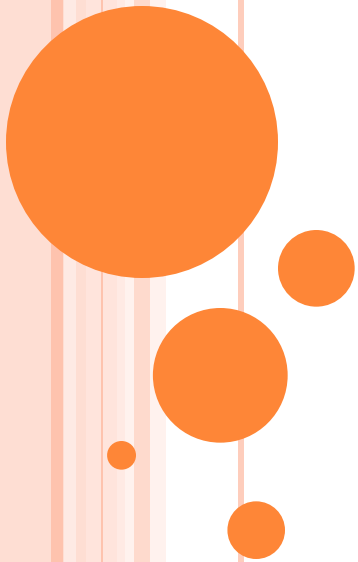


DATA STRUCTURES



DEFINITION

- Data may be organized in many different ways ;
- The logical or mathematical model of a particular organization of data is called **Data structure**.
- Example: array, stack, queue etc...



BASIC TERMINOLOGY IN DATA ORGANIZATION

- **Data:** Values or sets of values.

- **Data Item:** A single unit of data.

Example: A number or string.

- **Group Items:** Data items that are further divided into sub-items.

Example: Employee's full name divided into first name, middle name, and last name.

- **Elementary Items:** A data item that is not subdivided.

Example: A phone number.

- **Entity:** Something with certain attributes or properties that may be assigned values.

Example: Employee



- **Attribute:** A property or characteristic of an entity.

Example: Name, Age, Sex.

- **Values:** Specific information assigned to an attribute.

Example: "Bob" as the value of the Name attribute.

- **Entity Set:** A collection of entities with similar attributes.

- **Example:** All employees in a company, where each employee has attributes like Name, Age, and Sex.

- **Information:** Data that has been processed or given meaning.

- **Example:** "Bob is 30 years old" as meaningful information.



- **Field:** A single elementary unit of information representing an attribute of an entity.
Example: A field for "Name", "Age", etc.
- **Record:** A collection of field values representing an entity.
Example: A complete employee record with Name, Age, Sex.
- **File:** A collection of records for entities in a given entity set.
Example: A file containing records of all employees.
- **Primary Key:** A field whose value uniquely identifies a record in a file.
Example: Employee ID.



○ **Types of Records in a File**

- **Fixed Length Records:**

- Same number of data items and space for each record.
- Example: A file with records that all have a name, age, and gender, each using the same space.

- **Variable Length Records:**

- Different lengths for each record.
- Example: A file with employee records where some have additional attributes like phone number or department.

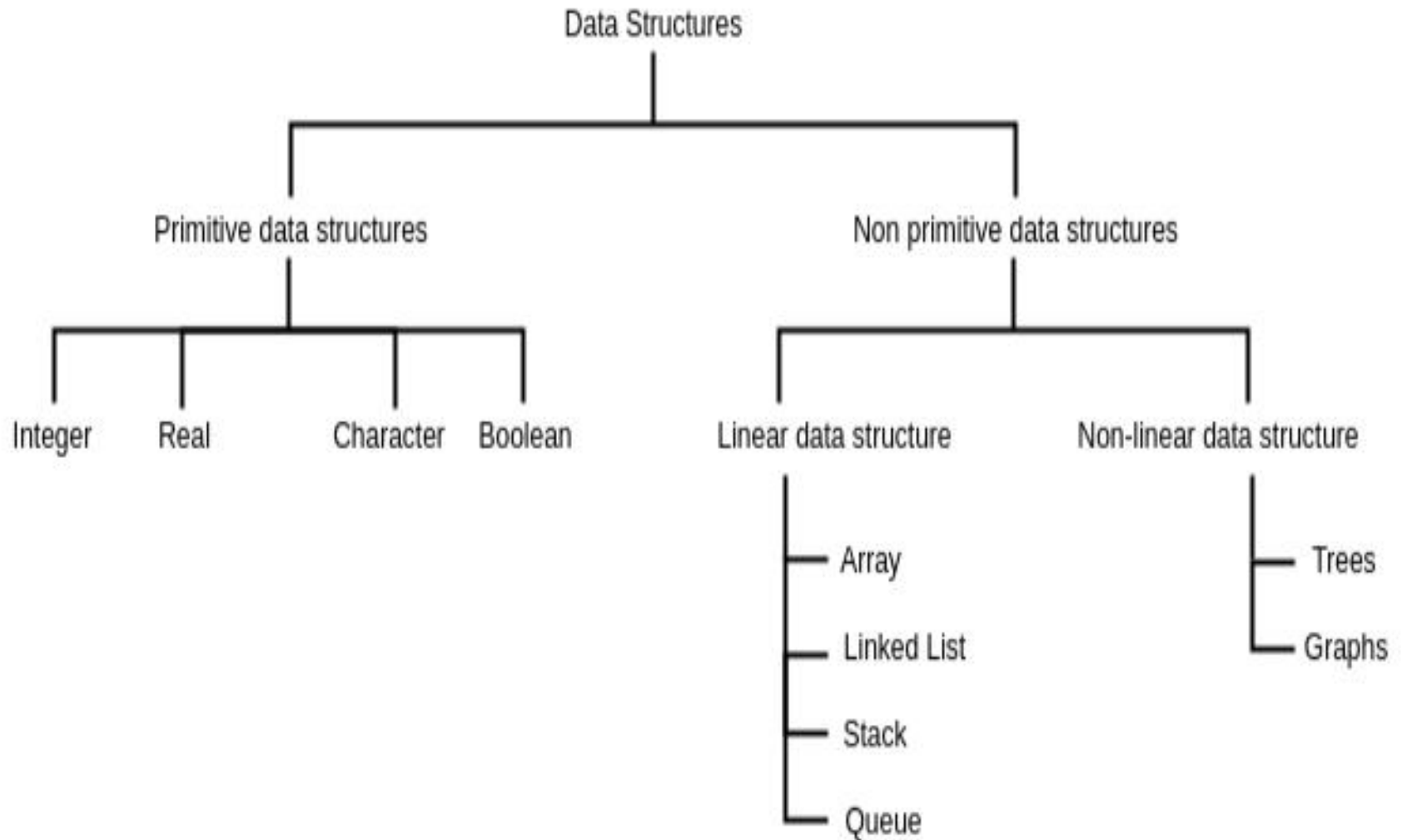


CLASSIFICATION OF DATA STRUCTURE

- Data structure are normally divided into two broad categories:
 - Primitive Data Structure
 - Non-Primitive Data Structure



CLASSIFICATION OF DATA STRUCTURE



PRIMITIVE DATA STRUCTURE

- **Definition:** Basic types of data that cannot be divided further.
- **Examples:**
 - **Integer:** Whole numbers (e.g., 1, 2, 3).
 - **Real:** Floating-point numbers (e.g., 3.14, 2.5).
 - **Character:** Single alphabet or symbol (e.g., 'A', 'B', 'c').
 - **Boolean:** True or False values.



NON-PRIMITIVE DATA STRUCTURE

- **Definition:** These are more complex data structures made by combining primitive data types.
- **Types:**
 - **Linear Data Structures**
 - **Non-Linear Data Structures**



Linear Data Structures

- **Definition:** Data elements are arranged sequentially or linearly.
- **Examples:**
 - **Array:** Collection of elements stored at contiguous memory locations.
 - **Linked List:** A sequence of elements, where each element points to the next.
 - **Stack:** A list where elements are added or removed from the same end (LIFO – Last In, First Out).
 - **Queue:** A list where elements are added at one end and removed from the other (FIFO – First In, First Out).



Non-Linear Data Structures

- **Definition:** Data elements are not arranged sequentially.
- **Examples:**
 - **Tree:** A hierarchical structure with a root node and subtrees.
 - **Graph:** A collection of nodes connected by edges, used to represent relationships between entities.



DATA STRUCTURE OPERATIONS

Operations on Linear Data Structures



Insertion



Deletion



Searching



Traversing



Sorting



Merging

- **Traversing:** accessing each record exactly once so that certain items in the record may be processed.
- **Searching:** finding the location of the record with a given key value or finding the locations of all records which satisfy one or more conditions.
- **Insertions:** adding s new record to a structure.
- **Deleting:** removing a record from a structure.
- **Sorting:** arranging the records in the some logical order.
- **Merging:** combining the records in two different sorted files into a single sorted file.

ALGORITHMIC NOTATION

- **Algorithm:** A finite, step-by-step list of well-defined instructions to solve a specific problem.
- **Steps:** Executed one after another, beginning with step 1.
- **Control Transfer:** Can use "goto step n" to jump to a different step.
- **Sequential Execution:** If multiple statements are in a single step, they are executed from left to right.
 - e.g Set $K:=1$, $Loc:=1$ and $max:=A[1]$



- **Exit:** Marks the end of the algorithm, similar to the STOP statement in FORTRAN or flowcharts.
- **Comments:** Each step may contain a comment in brackets, explaining the purpose of that step.
- Comment will usually appear at the beginning or the end of the step
- **Variable Names:**
 - Variable names will use capital letters for constants (e.g., MAX) and counters (eg., K, N).
 - Lowercase can be used for regular variables (e.g., n, value)
- **Assignment Statement:**

Use:= for assignments.
eg: MAX:=A[1]



- **Input/Output:**
 - **Input:** Use Read(variable names).
Eg:Read(A, B).
 - **Output:**Use Write(message and/or variable names).
Eg:Write('sum is', c).
- **Procedures :**Procedure is an independent algorithmic module which solves a particular problem.

Example: algorithm notation

Step 1 . START

Step 2 .[declare three integers] A, B , C.

Step 3 . [define values of A & C]SET A:=1,B:=3

Step 4 .C:=A+B .

Step 5 .WRITE:C

Step 6.EXIT.



Control structures

```
graph TD; A[Control structures] --> B[Sequential logic]; A --> C[Conditional /Selection logic]; A --> D[Iterative /Repetitive logic];
```

Sequential
logic

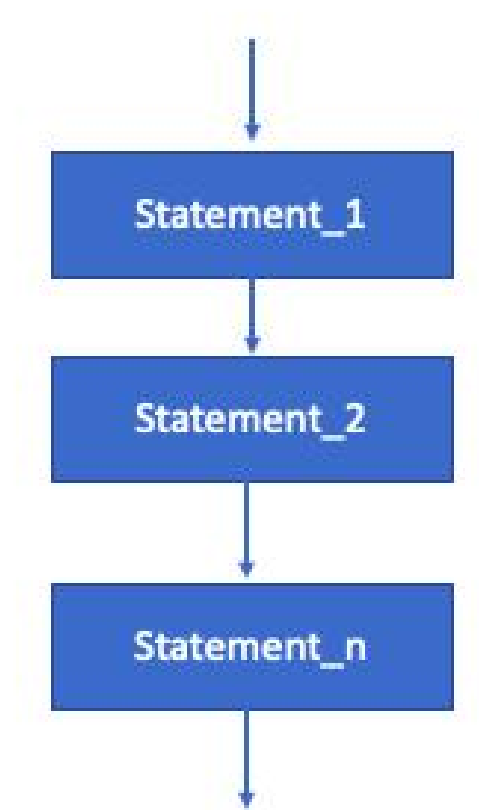
Conditional
/Selection
logic

Iterative
/Repetitive
logic



SEQUENTIAL LOGIC

- Instructions are executed in the sequential order.



Sequence Control Structure



SELECTION LOGIC

- Selection logic uses conditions to choose one module from several alternatives.
- The structures which implement this logic are called conditional structures or if structures.
- There are 3 types of conditional structures;
 - Single alternative
 - Double alternative
 - Multiple alternative:



Single alternative:

- This structure has the form

If condition, then:

[Module A]

[End of if structure]

Here, the if condition holds, then module A which consists of one or more statements is executed; otherwise Module A is skipped and control transfers to the next step of the algorithm.

eg: if $n \% 2 == 0$, then:

write: 'even no' n exit.

write:'odd' n exit.



DOUBLE ALTERNATIVE

- This structure has the form

If condition, then:

[Module A]

Else

[Module B]

[End of if structure]

if the condition holds, then module A is executed;
otherwise Module B is execute

if $A > B$, then:

write: 'largest is', A

else:

write: B

[end of if structure]

Exit.



MULTIPLE ALTERNATIVE:

- This structure has the form:

If condition(1), then:

[Module A1]

Else if condition(2), then:

[Module A2]

.

.

.

Else if condition(M), then:

[Module AM]

Else: [Module B]

[End of if structure]



ITERATIVE LOGIC

- Looping structures are used to repeat a set of instructions multiple times.
- There are two main types of loops:
 - **Repeat-For** and **Repeat-While**.

1. Repeat-For Loop

The **Repeat-For** loop runs a specific number of times.

It uses a variable (Like k) that starts at a certain value and increases by a fixed amount each time until it reaches the end value.

Repeat for k = 1 to 5 by 1:

Print k

[End of loop]

This loop will print the numbers from 1 to 5.



2. Repeat-While.

- The **Repeat-While** loop repeats instructions as long as a condition is true. When the condition becomes false, the loop stops.

Example:

Start with $k = 1$

Repeat while $k \leq 5$:

Print k

Increase k by 1

[End of loop]

This loop will also print numbers from 1 to 5, but the loop stops when k is greater than 5.



RECURSION

- Recursion is a method of solving problems by breaking them down into smaller versions of the same problem.
- It involves a function calling itself, either directly or indirectly, until a condition is met.
- A recursive procedure must have the following two properties:
 - There must be a criteria - "base criteria" are the conditions that stop a function from calling itself in recursion.
 - Each time the function calls itself, it should get closer to reaching the condition that stops the recursion.



RECURSION TECHNIQUE EXAMPLES

- Factorial.
- Fibonacci sequence.
- Towers of Hanoi.



FACTORIAL.

- The product of the positive integers from 1 to n , inclusive, is called **n factorial** and is usually denoted by **$n!$** .
- Mathematically, expressed as:

$$n! = 1 \times 2 \times 3 \times \cdots \times n$$

$$0! = 1$$

$$1! = 1$$

$$2! = 1 * 2 = 2$$

$$3! = 1 * 2 * 3 = 6$$

$$4! = 1 * 2 * 3 * 4 = 24$$

$$5! = 1 * 2 * 3 * 4 * 5 = 120$$

and so on. Observe that:

$$5! = 5 * 4! = 5 * 24 = 120$$

$$6! = 6 * 5! = 6 * 120 = 720$$

is true for every positive integer n , that is:

$$\mathbf{n! = n * (n-1)!}$$



Iterative:

PROCEDURE FACTORIAL(FACT, N)

1. If $N = 0$, then

$FACT = 1$

 RETURN

2. Set $FACT = 1$

3. Repeat for $K = 1$ to N :

$FACT = K * FACT$

4. RETURN



Recursive :

Procedure : FACTORIAL(FACT, N)

This procedure calculates $N!$ and returns the value in the variable FACT.

1. If $N = 0$, then

$FACT = 1$

 RETURN

2. Call FACTORIAL(FACT, $N - 1$)

3. Set $FACT = N * FACT$

4. RETURN



FIBONACCI SEQUENCE.

the Fibonacci sequence is usually denoted by $F_0, F_1, \dots$) is as follows:

0, 1, 1, 2, 3, 5, 8, 13, -----

That is, $F_0 = 0$ and $F_1 = 1$ and each succeeding term is the sum of the two preceding terms. For example the next two terms of the sequence are

$$8 + 13 = 21$$

A formal definition is

If $n = 0$ or $n = 1$ then $f_n = n$

If $n > 1$ then $f_n = f_{n-2} + f_{n-1}$ this is an other example of recursive definition.

The value of f_n is defined in term of smaller value of n which are closer to the base value. Accordingly this function is well-defined.



Procedure for finding the n th term F_n of Fibonacci sequence as follows.

FIBONACCI (FIB, N)

This procedure calculates F_n , and return values in the first parameter FIB.

1. If $N=0$ or $N = 1$, then set $FIB = N$ and return
2. Call FIBONACCI (FIB_A, $N-2$)
3. Call FIBONACCI (FIB_B, $N-1$)
4. Set $FIB = FIB_A + FIB_B$
5. Return



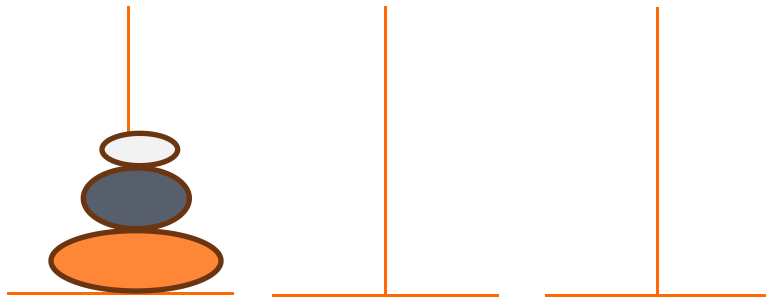
TOWERS OF HANOI.

- The Tower of Hanoi is a classic puzzle with three pegs and a stack of different-sized disks.
- The goal is to move the entire stack from one peg to another.
- The minimum number of moves is $2^n - 1$, where n is the number of disks



Rules:

- **One at a Time:** Only one disk can be moved at a time.
- **Top Disk Only:** Only the top disk on a peg can be moved.
- **Larger over Smaller:** A larger disk cannot be placed on top of a smaller disk.



TOWER(N, BEG, AUX, END)

This procedure gives a recursive solution to the Tower of Hanoi problem for N disks

1. IF N = 1, THEN:

a) Write: BEG --> END

b) Return

[End of IF structure]

2. Move N-1 disks from peg BEG to peg AUX.

Call TOWER(N-1, BEG, END, AUX).

3. Write BEG --> END.

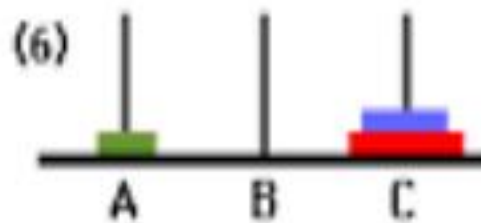
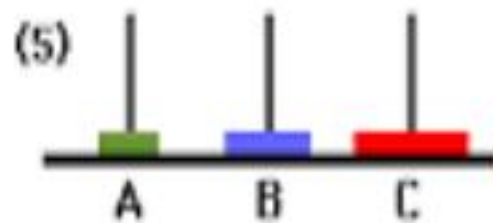
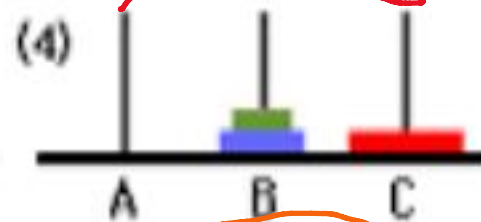
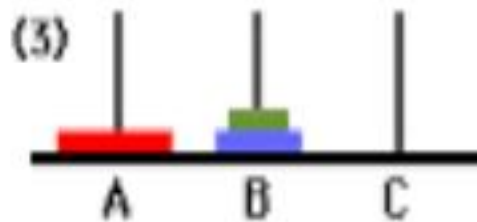
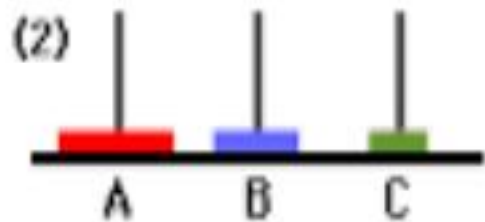
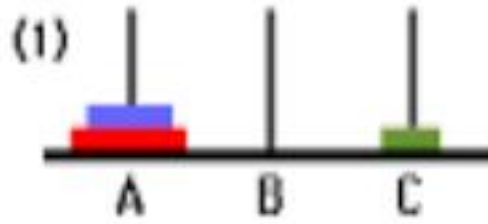
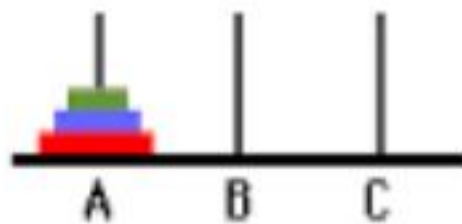
4. Move N-1 disks from peg AUX to peg END.

Call TOWER(N-1, AUX, BEG, END).

5. Return



3 DISKS



Here are the steps to solve the 3-disk Tower of Hanoi:

1. Move disk 1 from A to C:
2. Move disk 2 from A to B:
3. Move disk 1 from C to B:
4. Move disk 3 from A to C:
5. Move disk 1 from B to A:
6. Move disk 2 from B to C
7. Move disk 1 from A to C:



ARRAYS

- A linear array is a list of finite number n of homogeneous data elements (i.e data elements of the same type) such that:
 - The elements of the array are referenced respectively by an index set consisting of n consecutive numbers.
 - The elements are stored respectively in successive memory locations.
- The number n of elements is called the length or size of the array
 - $\text{Length} = \text{UB} - \text{LB} + 1$
 - UB --upper bound
 - LB --lower bound.
 - The elements of an array A may be denoted by the subscript notation
 - $A[1], A[2], \dots A[N]$.



REPRESENTATION OF LINEAR ARRAY IN MEMORY

- Let LA be a linear array in the memory of the computer
 - $\text{LOC}(\text{LA}[K])$ = address of the element $\text{LA}[K]$ of the array LA .
 - The element of LA are stored in the successive memory cells.

$$\text{LOC}(\text{LA}[K]) = \text{Base}(\text{LA}) + w(K - \text{lower bound})$$

- where w is the number of words per memory cell of the array LA [w is size of the data type]

- EXAMPLE1: Find the address for LA[6] Each element of the array occupy 1 byte.



$\text{LOC}(\text{LA}[K]) = \text{Base}(\text{LA}) + w(K - \text{lower bound})$

$\text{LOC}(\text{LA}[6]) = 200 + 1(6 - 1) = 205$

- EXAMPLE2: Find the address for LA[16] Each element of the array occupy 2 byte



TRAVERSING LINEAR ARRAYS:

- The following algorithm traverses a linear array.
 - Here LA is a linear array with lower bound LB and upper bound UB. This algorithm traverses an array A applying an operation PROCESS to each element of A.
- 1. **[initialize counter] Set $K:=LB$**
- 2. **Repeat steps 3 and 4 while $K \leq UB$**
- 3. **[visit element] Print $A[K]$**
- 4. **[increase counter] Set $K:=K+1$**
- 5. **[end of step 2 loop]**
- 6. **Exit**



INSERTING INTO LINEAR ARRAY

○ **Algorithm: inserting into a linear array**

Insert (LA, N, K, ITEM)

Here LA is a linear array with N elements. This algorithm inserts an element ITEM into the the Kth position in an array LA.

- 1. [Initialize counter] Set $J := N$.**
- 2. Repeat Steps 3 and 4 while $J \geq K$.**
- 3. [Move Jth element downward] Set $LA[J+1] := LA[J]$.**
- 4. [Decrease counter] Set $J := J-1$.**
- [End of Step 2 loop]**
- 5. [Insert element] Set $LA[K] := ITEM$.**
- 6. [Reset N] Set $N := N+1$**
- 7. Exit.**



DELETING FROM LINEAR ARRAY

○ Algorithm: deleting from an array

Delete (A, N, J, ITEM)

Here A is an array of N elements.

ITEM is an element which is to be deleted from the jth position of the array.

1. Set $ITEM := LA[K]$

2. Repeat for $J = K$ to $N-1$

[Move J + 1st element upward] Set $LA[J] := LA[J + 1]$

[End of loop]

3. [Reset the number N of elements in LA] Set $N := N - 1$

4. EXIT

MULTI DIMENSIONAL ARRAYS

- Multidimensional arrays are the one where elements are referenced by two or three subscripts.

type
arr_name[size1][size2]....[sizeN];

eg: `int a[10][20];`

Commonly used Multidimensional Arrays

1. Two-Dimensional Array (2D Array)
2. Three-Dimensional Array (3D Array)

TWO DIMENSIONAL ARRAYS(MATRICES)

- A **two-dimensional array** ($m \times n$ array) is a collection of **$m \times n$ data elements**, where each element is identified using two **subscripts (i, j)**.
- **Representation**
 - Elements are stored in a **rectangular format** with **m rows** and **n columns**.
 - The element at position **A[I, J]** is located in **row I** and **column J**.



○ Common Uses

- ✓ **Mathematics:** Matrices for calculations.
- ✓ **Business Applications:** Tables for organizing data.
- ✓ **Computer Science:** Image processing, spreadsheets, and databases.

○ Example:

A 3×3 matrix representation:

$$A = \begin{bmatrix} A[0, 0] & A[0, 1] & A[0, 2] \\ A[1, 0] & A[1, 1] & A[1, 2] \\ A[2, 0] & A[2, 1] & A[2, 2] \end{bmatrix}$$

REPRESENTATION OF 2D ARRAYS IN MEMORY

○ Memory Storage of a 2D Array

- A $m \times n$ array is stored in **sequential memory locations**.
- Storage can follow either **Row-Major Order** or **Column-Major Order**.

○ Memory Address Calculation

1 □ Row-Major Order (Stored row by row)

Formula:

$$\text{LOC}(A[J,K]) = \text{Base}(A) + w[n(J-1) + (K-1)]$$



2 □ Column-Major Order (Stored column by column)

Formula:

$$\text{LOC}(A[J,K]) = \text{Base}(A) + w[m(K-1) + (J-1)]$$

EXAMPLE: Consider a 25 x 4 array A. Suppose the $\text{Base}(A) = 200$ and $w = 4$. Suppose the programming store 2D array using row- major and column major. Compute $\text{LOC}(A[12,3])$



REPRESENTATION OF POLYNOMIALS USING ARRAY

- A polynomial of degree n can be represented using a **one-dimensional array**, where:
 - The **index** represents the **exponent** (power of x).
 - The **value at that index** represents the **coefficient**.

Consider the polynomial:

$$4x^4 + 3x^2 + 2x + 3$$

The highest degree (exponent) is 4.

The coefficient of x^4 is 4, x^2 is 3, x^1 is 2, and x^0 is 3.



Using an array:

Index (Exponent)	0	1	2	3	4
Coefficient	3	2	3	0	4

So, the array representation is:

$[3, 2, 3, 0, 4]$

We can use this representation for polynomial **addition, multiplication, and evaluation.**



Polynomial Operations Using Arrays

Once a polynomial is stored in an array, we can perform various operations like:

a) Polynomial Addition

To add two polynomials, align their exponents and sum the coefficients.

Example:

$$(4x^4 + 3x^2 + 2x + 3) + (x^3 + 5x + 2)$$

1. Represent the first polynomial: [3, 2, 3, 0, 4]
2. Represent the second polynomial: [2, 5, 0, 1, 0]

Result:

$$4x^4 + x^3 + 3x^2 + 7x + 5$$

Array representation: [5, 7, 3, 1, 4]



b) Polynomial Multiplication

To multiply two polynomials, multiply each term of the first polynomial by every term of the second polynomial and sum the results.

Example:

$$(2x + 3) \times (x^2 + 4)$$

Steps:

1. Multiply each term:

- $2x \times x^2 = 2x^3$
- $2x \times 4 = 8x$
- $3 \times x^2 = 3x^2$
- $3 \times 4 = 12$

2. Combine the terms:

$$2x^3 + 3x^2 + 8x + 12$$

Array representation: [12, 8, 3, 2]



DRAWBACKS OF USING ARRAYS FOR POLYNOMIAL REPRESENTATION

- **Large Exponent Requires a Large Array.**
 - If the polynomial has a very large exponent, the array must be large enough to accommodate it.
 - Example: $4x^{999}$ requires an array of size **1000**, even though it contains only one nonzero coefficient.
- **Wastage of Space**
 - Sparse polynomials (where most coefficients are zero) waste memory.
 - Example: $6x^{99} - 10$
 - An array of size **100** is needed, but only two values are used.
- **Fixed Array Size Problem**
 - If the array size is **too small**, it cannot accommodate higher exponents.



SPARSE MATRICES

- Sparse matrices are matrices that have a large number of zero elements.
- They are common in scientific computing, engineering, and optimization problems.
- General types of n -square sparse matrices,
 - Triangular Matrix
 - Lower triangular
 - Upper triangular
 - Tridiagonal Matrix



○ Lower Triangular Matrix

- A matrix where all elements above the main diagonal are zero.
- Nonzero elements are only on or below the main diagonal.

- Example:

$$\begin{bmatrix} a_{11} & 0 & 0 & 0 \\ a_{21} & a_{22} & 0 & 0 \\ a_{31} & a_{32} & a_{33} & 0 \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix}$$



○ Upper Triangular Matrix

- A matrix where all elements **below** the main diagonal are **zero**.
- Nonzero elements appear on or above the main diagonal.

- Example:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ 0 & a_{22} & a_{23} & a_{24} \\ 0 & 0 & a_{33} & a_{34} \\ 0 & 0 & 0 & a_{44} \end{bmatrix}$$



○ Tridiagonal Matrix

- A matrix where nonzero elements appear only:
 - On the **main diagonal**.
 - On the **diagonal immediately above** and **below** the main diagonal.

- Example:

$$\begin{bmatrix} a_{11} & a_{12} & 0 & 0 \\ a_{21} & a_{22} & a_{23} & 0 \\ 0 & a_{32} & a_{33} & a_{34} \\ 0 & 0 & a_{43} & a_{44} \end{bmatrix}$$



SORTING

- It refers to the operation of rearranging the elements of an array A so that they are in increasing order.
- For example, Suppose A has the following elements 8,4,19,2,7,13,5,16. After sorting the array A becomes 2,4,5,7,8,13,16,19.



SELECTION SORT

- Selection Sort is a **sorting algorithm** that repeatedly **selects the smallest (or largest) element** from the unsorted part of the array and **places it at the correct position** in the sorted part.
- 1. Start with the **first element** as the current position.
- 2. Find the **smallest element** in the unsorted portion of the array.
- 3. **Swap** it with the element at the current position.
- 4. Move to the **next position** and repeat steps 2–3.
- 5. Continue until the whole array is sorted.



SELECTION SORT

○ SELECTION (A,N)

This algm sorts the array A with N elements.

1. Repeat Steps 2 and 3 for $K=1,2,\dots,N-1$
2. Call MIN (A, K, N, LOC)
3. [Interchange A[K] and A[LOC]]
 SET TEMP:=A[K]
 A[K]:A[LOC] and
 A[LOC]:=TEMP
[End of step 1 loop]
4. Exit



○ MIN(A,K,N,LOC)

An array A is in memory .This procedure finds the location LOC of the smallest element among $A[K], A[K+1], \dots, A[N]$.

1. SET $MIN := A[K]$ and $LOC := K$ [Initialize Pointer]
2. Repeat for $J = K+1, K+2, \dots, N$:
 if $MIN > A[J]$ then: SET $MIN := A[J]$ &
 $LOC := J$ and $LOC := J$
[End of loop]
3. Return



Selection Sort

Pass	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]
K=1 LOC=4	77	33	44	11	88	22	66	55
K=2 LOC=6	11	33	44	77	88	22	66	55
K=3 LOC=6	11	22	44	77	88	33	66	55
K=4 LOC=6	11	22	33	77	88	44	66	55
K=5 LOC=8	11	22	33	44	88	77	66	55
K=6 LOC=7	11	22	33	44	55	77	66	88
K=7 LOC=4	11	22	33	44	55	66	77	88

Sorted	11	22	33	44	55	66	77	88
--------	----	----	----	----	----	----	----	----



BUBBLE SORT

- Bubble Sort is a **sorting algorithm** that repeatedly **compares adjacent elements** and **swaps them** if they are in the wrong order. After each pass, the **largest element “bubbles up”** to its correct position at the end of the array.
 1. Start from the **first element** of the array.
 2. *Compare it with the next element.*
 3. *If the current element is greater, swap them.*
 4. *Move to the next pair and repeat.*
 5. *Repeat the whole process for all elements until the array is sorted.*



BUBBLE SORT

(Bubble sort) BUBBLE(DATA,N)

1. Repeat steps 2&3 for $K=1$ to $N-1$
2. Set $PTR:=1$ [Initializes pass pointer PTR]
3. Repeat while $PTR \leq N-K$: [Executes pass]
 - a. If $DATA[PTR] > DATA[PTR+1]$,Then
Interchange $DATA[PTR]$ and $DATA[PTR+1]$.
[End of if structure]
 - b. Set $PTR:=PTR+1$
[End of inner loop]
[End of step 1 outer loop]
4. Exit.



INSERTION SORT

- Insertion Sort is a **sorting algorithm** that builds the final sorted array **one element at a time**.

It inserts each element into its **correct position** in the already sorted part of the array.

How it works:

1. Consider the **first element as sorted**.
2. Pick the **next element**.
3. Compare it with elements in the sorted part.
4. **Shift all elements greater than it** one position to the right.
5. **Insert the element** in the correct position.
6. Repeat steps 2–5 for all elements.

Sentinel is a **dummy value** (like $-\infty$) placed at the start of the array. It stops the inner loop automatically and avoids checking array bounds.



INSERTION SORT

INSERTION(A,N)

This algorithm sorts the array A with N elements.

1. Set $A[0] := -\infty$ [Initializes sentinel element]
2. Repeat Steps 3 to 5 for $K=2,3,\dots,N$.
3. Set $TEMP=A[K]$ and $PTR:=K-1$
4. Repeat while $TEMP < A[PTR]$
 - a) Set $A[PTR+1] := A[PTR]$
 - b) Set $PTR := PTR - 1$
5. Set $A[PTR+1] := TEMP$
6. Return



Example: Sort an array A with 8 elements
77, 33, 44, 11, 88, 66, 55

Pass	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]
K=1	$-\infty$	77	33	44	11	88	22	66	55
K=2	$-\infty$	77	33	44	11	88	22	66	55
K=3	$-\infty$	33	77	44	11	88	22	66	55
K=4	$-\infty$	33	44	77	11	88	22	66	55
K=5	$-\infty$	11	33	44	77	88	22	66	55
K=6	$-\infty$	11	33	44	77	88	22	66	55
K=7	$-\infty$	11	22	33	44	77	88	66	55
K=8	$-\infty$	11	22	33	44	66	77	88	55
Sorted	$-\infty$	11	22	33	44	55	66	77	88



MERGE SORT

- It is a sorting algorithm that uses the divide , conquer & combine algorithmic paradigm.

1. Divide: Partitioning the n element array to be sorted into 2 sub-arrays of $n/2$ elements.

If a is an array containing zero or one element, then it is already sorted.

If there are more elements in the array divide A into 2 sub-arrays A_1 & A_2 , each containing about half of the elements of A .

2. Conquer: Means sorting the 2 sub-arrays recursively using mergesort.

3. Combine: Means merging the 2 sorted sub-arrays of size $n/2$ to produce the sorted array of n elements.



MERGE SORT ALGORITHM

MERGE_SORT(ARR, BEG, END)

Step 1: If $BEG < END$

Set $MID = (BEG + END) / 2$

Call MERGE_SORT(ARR, BEG, MID)

Call MERGE_SORT(ARR, MID + 1, END)

Call MERGE(ARR, BEG, MID, END)

[End of If]

Step 2: Exit



MERGE (ARR, BEG, MID, END)

Step 1: [Initialize] SET $I = \text{BEG}$, $J = \text{MID} + 1$, and $\text{INDEX} = 0$.

Step 2: Repeat while ($I \leq \text{MID}$) and ($J \leq \text{END}$)

If $\text{ARR}[I] < \text{ARR}[J]$

Set $\text{TEMP}[\text{INDEX}] = \text{ARR}[I]$.

Set $I = I + 1$.

Else

Set $\text{TEMP}[\text{INDEX}] = \text{ARR}[J]$.

Set $J = J + 1$.

[END OF IF]

Set $\text{INDEX} = \text{INDEX} + 1$.

[End of loop]

Step 3: [Copy the remaining elements of right sub-array, if any]

If $I > \text{MID}$

Repeat while $J \leq \text{END}$

Set $\text{TEMP}[\text{INDEX}] = \text{ARR}[J]$.

Set $\text{INDEX} = \text{INDEX} + 1$, Set $J = J + 1$.

[End of loop]

[Copy the remaining elements of left sub-array, if any]

Else

Repeat while $I \leq \text{MID}$

Set $\text{TEMP}[\text{INDEX}] = \text{ARR}[I]$.

Set $\text{INDEX} = \text{INDEX} + 1$, Set $I = I + 1$.

[End of loop]

[End of If]

Step 4: [Copy the contents of TEMP back to ARR] Set $K = 0$.

Step 5: Repeat while $K < \text{INDEX}$

Set $\text{ARR}[K] = \text{TEMP}[K]$.

Set $K = K + 1$.

[End of loop]

Step 6: End.



QUICK SORT

- Quick Sort is a **fast and efficient sorting algorithm** that uses the **divide-and-conquer** approach to sort an array or list. It works by selecting a **pivot element** and arranging the array so that:
 - All elements **less than the pivot** come **before** it
 - All elements **greater than the pivot** come **after** it
- Then it **recursively applies the same process** to the left and right sub-arrays until the array is fully sorted.



Steps of Quick Sort

- **Choose a Pivot:**

Select an element from the array (commonly the first, last, or middle element).

- **Partition the Array:**

Rearrange elements so that all smaller elements go to the left of the pivot, and all larger elements go to the right. Place the pivot in its **correct position**.

- **Recursively Sort Sub-Arrays:**

- Apply Quick Sort to the **left sub-array** (elements smaller than pivot)
- Apply Quick Sort to the **right sub-array** (elements larger than pivot)

- **Stop Condition:**

When a sub-array has **0 or 1 element**, it is already sorted.



PROCEDURE PARTITION(A, BEG, END, LOC)

1. [Initialize] SET LEFT = BEG, RIGHT = END, PIVOT = A[BEG],
2. WHILE TRUE:
 - a. WHILE $A[\text{LEFT}] \leq \text{PIVOT}$ AND $\text{LEFT} < \text{RIGHT}$ OR $\text{LEFT} == \text{BEG}$:
 - i. INCREMENT LEFT ($\text{LEFT} = \text{LEFT} + 1$)
 - b. WHILE $A[\text{RIGHT}] > \text{PIVOT}$:
 - i. DECREMENT RIGHT ($\text{RIGHT} = \text{RIGHT} - 1$)
 - c. IF $\text{LEFT} \geq \text{RIGHT}$:
 - i. BREAK LOOP
 - d. SWAP $A[\text{LEFT}]$ WITH $A[\text{RIGHT}]$
3. SWAP $A[\text{BEG}]$ WITH $A[\text{RIGHT}]$ (Place Pivot in Correct Position)
4. SET $\text{LOC} = \text{RIGHT}$
5. RETURN LOC
6. END PROCEDURE

```
PROCEDURE QUICK_SORT(A, BEG, END)
1. IF BEG < END:
  a. CALL PARTITION(A, BEG, END, LOC)
  b. CALL QUICK_SORT(A, BEG, LOC - 1)
  c. CALL QUICK_SORT(A, LOC + 1, END)
2. END PROCEDURE
```

